

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Технології розподілених систем та паралельних обчислень»

Робота №4

Тема «Функції обміну в MPI»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Дьячук Т.С.

2025

МЕТА

Метою роботи є продовжити вивчення функцій обміну бібліотеки MPI, зокрема, функцій колективного обміну. Освоїти деякі прийоми їх використання для розподілу даних та обчислень між паралельними процесорами.

ЗАВДАННЯ

4.2.1 Завдання 1

Програма виконує паралельне обчислення суми ряду $\sum_{i=0}^{N-1} a_i x_i$. Для цього до кожного з P процесів за допомогою функції *MPI_Scatter()* передається відповідна частина масива чисел a_i та x_i , кожна з яких складається з $M=N/P$ елементів. Процеси обчислюють часткові суми ряду (змінна *sum*), які далі за допомогою функції *MPI_Reduce()* приводяться до загальної суми (змінна *total*).

4.2.1.1 Створити програму обчислення суми ряду.

$\sum_i a_i \cdot \sin(x_i)$, де $i=0,1,\dots,N$; $N=240\,000$, $x_i = 0.0001 * I$, a_i – випадкові дійсні числа

в діапазоні $(-1, 1)$. Функцію $\sin(x_i)$ обчислювати за допомогою розкладання її в ряд Тейлора: $\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots + (-1)^n \frac{x^{2n+1}}{(2 \cdot n + 1)!} + \dots$

Обчислення кожного значення функції $\sin(x_i)$ можна організувати за наступним алгоритмом:

а) $y = x$; $s = y$; $k = 1$;

б) $y = - (x^2 / (k+1)*(k+2)) * y$;

в) $s = s + y$; $k = k + 2$;

г) якщо $k \leq K$, то перехід на п. б), інакше — кінець.

Для обчислення синуса в програмі створити окрему функцію, K нехай дорівнює 500.

4.2.1.2 Виміряйте час виконання розробленої програми на одному, двох, трьох та чотирьох процесорах для $N=240000$ та визначте прискорення обчислень для цих випадків. Дані занесіть до таблиці 4.1 (форми таблиць наведені нижче). Намалюйте графік залежності прискорення від кількості процесорів.

4.2.1.3 Дослідити та намалювати графіки залежності прискорення від кількості членів ряду при обчисленнях на чотирьох процесорах, змінюючи кількість членів ряду від 20000 до 240000 з дискретністю 20000. Вимірювання виконуйте 3 рази з інтервалом між вимірюваннями 15 хвилин, результати вимірювань занесіть до таблиці 4.2.

4.2.1.4 Додайте до програми код для вимірювання часу, який витрачається на виконання двох функцій `MPI_Scatter()`, дослідіть та намалюйте графіки залежності цього часу від довжини масиву, що розсилається при виконанні задачі на чотирьох процесорах. Результати занесіть до таблиці 4.3.

Таблиця 4.1 – Залежність прискорення від кількості процесорів ($N=240000$, номер робочого місця №19)

Дата та час	Число процесорів P	T ₁	T _p	S
-------------	--------------------	----------------	----------------	---

Таблиця 4.2 – Залежність прискорення від кількості членів ряду при $P=4$

Дата та час	N	T ₁	T ₄	S
-------------	---	----------------	----------------	---

Таблиця 4.3 – Залежність швидкості розсилання від довжини масиву при $P=4$

Дата та час	N	T _{Scatter}	S (байт / с)
-------------	---	----------------------	--------------

4.2.2 Завдання 2

$$\sum_{i=1}^N a_i x_i^i$$

Постановка задачі. Нехай необхідно обчислити суму ряду , де N - досить велике число та обчислення слід розподілити між P процесорами, причому, степінь чисел x_i повинна обчислюватись шляхом їх перемноження. Якщо в кожному процесорі обчислювати частину суми, яка складається з N/P елементів вихідного ряду, то це приведе до нерівномірного навантаження процесорів. Наприклад, якщо N=30, P=3, то розбивання ряду на три частини по N/P елементів

$$S = \sum_{i=1}^{10} a_i x_i^i + \sum_{i=11}^{20} a_i x_i^i + \sum_{i=21}^{30} a_i x_i^i.$$

буде: . В результаті перший процесор виконає 55, другий – 155, а третій – 255 множень. В програмі реалізовано рівномірний розподіл навантаження обчислень цієї модельної задачі наступним чином.

В другому елементі ряду, що розглядається, необхідно виконати одне множення, в третьому – два і т.д. Загальна кількість множень будь-якого "відрізка" ряду з M елементів утворюють арифметичну прогресію, отже:

$$S_M = n_1 \cdot M + \frac{(M - 1) \cdot M}{2}$$

де n_1 - номер першого елемента ділянки.

Тоді кількість операцій множення, яку повинний виконати кожний процесор, можна визначити за формулою:

$$S_N = \frac{1}{P} \left(1 \cdot N + \frac{(N - 1) \cdot N}{2} \right) = \frac{N(N + 1)}{P}$$

Для знаходження кількості операцій множення, яку повинний виконати перший процесор, складемо рівняння:

$$n_1 \cdot M_1 + \frac{(M_1 - 1) \cdot M_1}{2} = \frac{N(N + 1)}{P}$$

де $n_1=1$.

Розв'язуючи це квадратне рівняння, знайдемо значення M_1 – кількість елементів першого відрізка. Початком другого відрізка, очевидно, буде елемент з номером $n_2 = n_1 + M_1$, тоді кількість елементів другого відрізка, призначеного для обчислення на другому процесорі, можна знайти аналогічно, розв'язуючи рівняння:

$$n_2 \cdot M_2 + \frac{(M_2 - 1) \cdot M_2}{2} = \frac{N(N + 1)}{P}$$

і т.д.

З тим, щоб врахувати незначну неточність у визначенні довжин відрізків, пов'язану з цілочисленним розподіленням, кількість елементів відрізка для останнього процесора знайдемо із співвідношення:

$$M_p = N - M_1 - M_2 - \dots - M_{p-1}$$

Реалізація програми. Для розподілення різної кількості елементів між процесами використовується функція `MPI_Scatterv()`, яку описано вище.

4.2.2.1 Для перевірки результату паралельного обчислення додайте до програми код, який обчислює суму даного ряду тільки в нульовому процесі та використовує для обчислення степеня бібліотечну функцію.

4.2.2.2 Використовуючи наведені вище програми, розробіть власну програму, в якій між паралельними процесами розподіляється однакова кількість членів ряду $\sum_{i=1}^N a_i x_i^i$

4.2.2.3 Виміряйте час роботи програми для трьох, досить великих значень кількості членів ряду і порівняйте його з часом вирішення цієї задачі за допомогою вихідної програми даного завдання для двох, трьох та чотирьох процесорів. Результати наведіть у вигляді таблиці.

ВИКОНАННЯ

4.2.1.1 Програма

Завдання

Створити програму обчислення суми ряду.

$\sum_i a_i \cdot \sin(x_i)$, де $i=0,1,\dots,N$; $N=240\,000$, $x_i = 0.0001 * I$, a_i – випадкові дійсні числа

в діапазоні $(-1, 1)$. Функцію $\sin(x_i)$ обчислювати за допомогою розкладання її в ряд Тейлора:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots + (-1)^n \frac{x^{2n+1}}{(2 \cdot n + 1)!} + \dots$$

Обчислення кожного

значення функції $\sin(x_i)$ можна організувати за наступним алгоритмом:

а) $y = x$; $s = y$; $k = 1$;

б) $y = -(x^2 / (k+1)*(k+2)) * y$;

в) $s = s + y$; $k = k + 2$;

г) якщо $k \leq K$, то перехід на п. б), інакше — кінець.

Для обчислення синуса в програмі створити окрему функцію, K нехай дорівнює 500.

Код

```
from mpi4py import MPI
import random
import numpy as np

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

def custom_sin(x, this_K=500):
    y = x
    s = y
    k = 1
    while k <= this_K:
        y = -((x**2) / ((k + 1) * (k + 2))) * y
        s = s + y
        k = k + 2
    return s

N = 240_000
K = 500
```

```

chunk_size = N // size

a_full = None
x_full = None
if rank == 0:
    a_full = np.array([random.uniform(-1, 1) for _ in range(N)],
dtype="d")
    x_full = np.array([0.0001 * i for i in range(N)], dtype="d")

a_local = np.empty(chunk_size, dtype="d")
x_local = np.empty(chunk_size, dtype="d")

t_scatter_start = MPI.Wtime()
comm.Scatter(a_full, a_local, root=0)
comm.Scatter(x_full, x_local, root=0)
t_scatter_end = MPI.Wtime()

start_time = MPI.Wtime()

local_sum = 0.0
for i in range(chunk_size):
    local_sum += a_local[i] * custom_sin(x_local[i], this_K=K)

total_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)

end_time = MPI.Wtime()

if rank == 0:
    print(f"Proceses amount: {size}")
    print(f"Total sum: {total_sum}")
    print(f"Running time (Tp): {end_time - start_time:.4f} sec")
    print(f"Scatter time: {t_scatter_end - t_scatter_start:.6f} sec")

```

Результати

```

Proceses amount: 1
Total sum: -44.749436265304396
Running time (Tp): 24.8317 sec
Scatter time: 0.001168 sec

```

4.2.1.2 Значення

Завдання

Виміряйте час виконання розробленої програми на одному, двох, трьох та чотирьох процесорах для $N=240\,000$ та визначте прискорення обчислень для цих випадків. Дані занесіть до таблиці 4.1. Намалюйте графік залежності прискорення від кількості процесорів.

Код

```
import matplotlib.pyplot as plt

P_values = [1, 2, 3, 4] # Кількість процесорів
Tp_values = [24.6048, 12.5789, 8.4814, 6.6128]

# 1. Розрахунок прискорення S
T1 = Tp_values[0]
S_values = [T1 / Tp for Tp in Tp_values]

# 2. Розрахунок ідеального прискорення (для порівняння)
ideal_S = P_values

# 3. Налаштування графіка
plt.figure(figsize=(10, 6))
plt.plot(P_values, S_values, "ro-", label="Реальне прискорення (S)",
         linewidth=2)
plt.plot(P_values, ideal_S, "b--", label="Ідеальне прискорення (S=P)",
         alpha=0.7)

# Додавання підписів значень над точками
for i, txt in enumerate(S_values):
    plt.annotate(
        f"{txt:.2f}",
        (P_values[i], S_values[i]),
        textcoords="offset points",
        xytext=(0, 10),
        ha="center",
    )

# Оформлення осей та заголовка
plt.title("Залежність прискорення від кількості процесорів (N=240000)",
         fontsize=14)
plt.xlabel("Кількість процесорів (P)", fontsize=12)
plt.ylabel("Прискорення (S)", fontsize=12)
plt.xticks(P_values)
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()

# Збереження та показ
plt.savefig("speedup_graph.png")
print("Графік збережено у файл 'speedup_graph.png'")
plt.show()
```

Результати

```
$ mpiexec -n 1 python 1.py
Proceses amount: 1
Total sum: 146.733741072558
Running time (Tp): 24.6048 sec
```


Scatter time: 0.000619 sec

```
$ mpiexec -n 2 python 1.py
Proceses amount: 2
Total sum: -151.31365322656524
Running time (Tp): 12.5789 sec
Scatter time: 0.009543 sec
```

```
$ mpiexec -n 3 python 1.py
Proceses amount: 3
Total sum: -314.8372543611528
Running time (Tp): 8.4814 sec
Scatter time: 0.001679 sec
```

```
$ mpiexec -n 4 python 1.py
Proceses amount: 4
Total sum: -363.7585435844192
Running time (Tp): 6.6128 sec
Scatter time: 0.008873 sec
```

Таблиця

Значення S (прискорення) виміряно такою формулою:

$$S = \frac{T_1}{T_p}$$

Таблиця 4.1 – Залежність прискорення від кількості процесорів

Дата та час	Число процесорів P	T ₁	T _p	S
13.02.2026 17:38	1	24.6048	24.6048	1
13.02.2026 17:40	2		12.5789	1.95603749
13.02.2026 17:41	3		8.4814	2.90103049
13.02.2026 17:43	4		6.6128	3.72078393

Графік

Графік залежності наведено нижче на рисунку:

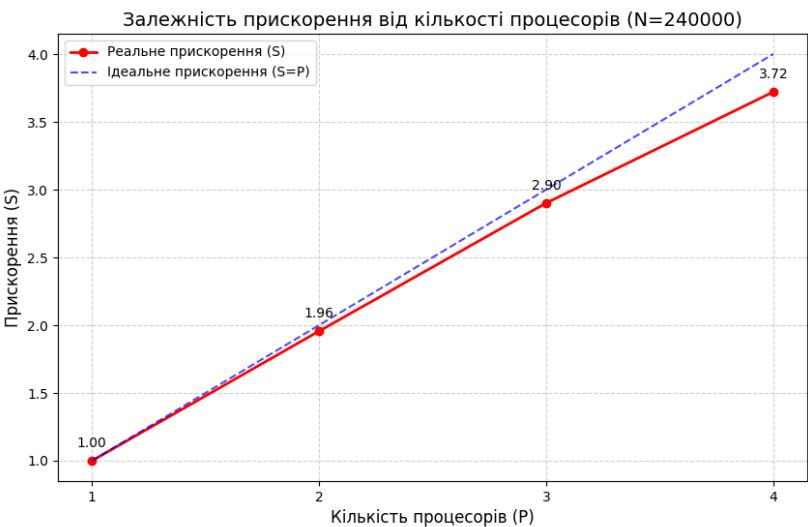


Рисунок 1.1 — Графік залежностей

4.2.1.3 Прискорення

Завдання

Дослідити та намалювати графіки залежності прискорення від кількості членів ряду при обчисленнях на чотирьох процесорах, змінюючи кількість членів ряду від 20 000 до 240 000 з дискретністю 20000.

Вимірювання виконуйте 3 рази з інтервалом між вимірюваннями 15 хвилин, результати вимірювань занесіть до таблиці 4.2.

Код обчислень

```
from mpi4py import MPI
import numpy as np
import random

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

def custom_sin(x, K=500):
    y = x
    s = y
    k = 1
    while k <= K:
        y = -((x**2) / ((k + 1) * (k + 2))) * y
        s = s + y
        k = k + 2
    return s

# Experiment settings
N_values = list(range(20_000, 240_001, 20_000))
P = size

if rank == 0:
    print(
        f"{'N':>10} | {'Attempt 1':>14} | {'Attempt 2':>14} | {'Attempt 3':>14} | {'Average Tp':>14}"
    )
    print("-" * 80)

for N in N_values:
    attempts = []
```

```

# Number of tests
for _ in range(3):
    comm.Barrier()
    start_time = MPI.Wtime()

    # Chunking the work
    chunk_size = N // size
    local_sum: float = 0.0

    for i in range(rank * chunk_size, (rank + 1) * chunk_size):
        a_i = random.uniform(-1, 1)
        x_i = 0.0001 * i
        local_sum += a_i * custom_sin(x_i)

    total_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)
    end_time = MPI.Wtime()

    attempts.append(end_time - start_time)

if rank == 0:
    average_tp = sum(attempts) / 3
    print(
        f"{N:10d} | {attempts[0]:14.4f} | {attempts[1]:14.4f} |
{attempts[2]:14.4f} | {average_tp:13.4f}"
    )

```

Таблиця

Таблиця 4.2 – Залежність прискорення від кількості членів при P=4

Дата та час	N	T ₁	T ₄	S
13.02.2026 19:37	20 000	1.3116	0.3608	3,6352549
13.02.2026 19:44	40 000	2.6094	0.7315	3,5671907
13.02.2026 19:44	60 000	3.9110	1.0885	3,5930179
13.02.2026 19:45	80 000	5.2144	1.4619	3,5668650
13.02.2026 19:46	100 000	6.4986	1.7941	3,6222061
13.02.2026 19:46	120 000	7.8133	2.1824	3,5801411
13.02.2026 19:46	140 000	9.0948	2.5097	3,6238594
13.02.2026 19:47	160 000	10.3897	2.9134	3,5661769
13.02.2026 19:47	180 000	11.6800	3.3082	3,5306208
13.02.2026 19:48	200 000	12.9829	3.6003	3,6060606
13.02.2026 19:48	220 000	14.3236	3.9487	3,6274216
13.02.2026 19:49	240 000	15.6334	4.4261	3,5320937

Графік

Графік залежності прискорення від розміру задачі наведено нижче на рисунку:



Рисунок 1.2 — Графік залежностей

Код графіка

```
import matplotlib.pyplot as plt

N_list = [
    20000,
    40000,
    60000,
    80000,
    100000,
    120000,
    140000,
    160000,
    180000,
    200000,
    220000,
    240000,
]

S_N = [
```

```

3.6352549,
3.5671907,
3.5930179,
3.5668650,
3.6222061,
3.5801411,
3.6238594,
3.5661769,
3.5306208,
3.6060606,
3.6274216,
3.5320937,
]

plt.figure(figsize=(10, 5))
plt.plot(N_list, S_N, "gs-", label="Прискорення S(N) на P=4")
plt.xlabel("Кількість членів ряду (N)")
plt.ylabel("Прискорення (S)")
plt.title("Залежність прискорення від розміру задачі (N)")
plt.grid(True)
plt.legend()
plt.show()

```

4.2.1.4 Час Scatter

Завдання

Додайте до програми код для вимірювання часу, який витрачається на виконання двох функцій `MPI_Scatter()`, дослідіть та намалюйте графіки залежності цього часу від довжини масиву, що розсилається при виконанні задачі на чотирьох процесорах. Результати занесіть до таблиці 4.3.

Таблиця

Для обчислення колонки *S* було використано таку формулу:

$$S = \frac{16 \cdot N}{T_{scatter}}$$

Таблиця 4.3 – Залежність швидкості розсилання від довжини масиву при $P=4$

Дата та час	N	T _{Scatter}	S (байт / с)
14.02.2026 10:37	20 000	0.00018030	1 774 819 744,86966
14.02.2026 10:38	40 000	0.00014770	4 333 107 650,643196
14.02.2026 10:38	60 000	0.00018110	5 300 938 708,2275
14.02.2026 10:38	80 000	0.00032590	3 926 142 153,84615
14.02.2026 10:39	100 000	0.00029120	5 494 505 494,5055
14.02.2026 10:39	120 000	0.00043010	4 464 078 121,3671
14.02.2026 10:39	140 000	0.00044610	5 021 295 673,6158
14.02.2026 10:39	160 000	0.00045490	5 627 610 463,8382
14.02.2026 10:39	180 000	0.00038570	7 466 943 220,1193
14.02.2026 10:40	200 000	0.00052980	6 040 015 100,0378
14.02.2026 10:40	220 000	0.00071900	4 895 688 456,1892
14.02.2026 10:40	240 000	0.00055660	6 899 029 823,931

Графік

Графік залежностей часу комунікацій від довжини масиву наведено нижче:



Рисунок 1.3 — Графік залежностей

4.2.2.1 Один процес

Завдання

Для перевірки результату паралельного обчислення додайте до програми код, який обчислює суму даного ряду тільки в нульовому процесі та використовує для обчислення степеня бібліотечну функцію.

Код

```
from mpi4py import MPI
import numpy as np
import math

def solve_quadratic(n1, target_load):
    a, b, c = 0.5, n1 - 0.5, -target_load
    discriminant = b**2 - 4 * a * c
    if discriminant < 0:
        return 0
    m = (-b + math.sqrt(discriminant)) / (2 * a)
    return int(round(m))
```



```

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Параметри задачі
N = 1000
x_val = 1.001

sendcounts = np.zeros(size, dtype=int)
displs = np.zeros(size, dtype=int)
a_full = None

if rank == 0:
    # Генеруємо дані тільки на головному процесі
    a_full = np.random.rand(N).astype(np.float64)

    # Розрахунок рівномірного навантаження
    total_ops = N * (N + 1) / 2
    target_load = total_ops / size

    current_n1, total_assigned = 1, 0
    for i in range(size - 1):
        m_i = solve_quadratic(current_n1, target_load)
        if total_assigned + m_i > N:
            m_i = max(0, N - total_assigned)
        sendcounts[i] = m_i
        displs[i] = total_assigned
        total_assigned += m_i
        current_n1 += m_i
    sendcounts[size - 1] = N - total_assigned
    displs[size - 1] = total_assigned

    # Розповсюджуємо параметри
    local_n = comm.scatter(sendcounts, root=0)
    local_displ = comm.scatter(displs, root=0)

    # Передача сегментів масиву коефіцієнтів
    local_a = np.empty(local_n, dtype=np.float64)
    comm.Scatterv([a_full, sendcounts, displs, MPI.DOUBLE], local_a,
root=0)

    # --- ПАРАЛЕЛЬНЕ ОБЧИСЛЕННЯ ---
    local_sum = 0.0
    for i in range(local_n):
        global_idx = local_displ + i + 1
        # Обчислення степеня шляхом перемноження (за умовою задачі)
        power_val = 1.0
        for _ in range(global_idx):
            power_val *= x_val
        local_sum += local_a[i] * power_val

    parallel_res = comm.reduce(local_sum, op=MPI.SUM, root=0)

    # --- ПЕРЕВІРКА НА НУЛЬОВОМУ ПРОЦЕСІ ---
    if rank == 0:
        print("-" * 30)

```

```

print(f"Результат MPI ({size} проц.): {parallel_res:.10f}")

# Послідовне обчислення з бібліотечною функцією
sequential_sum = 0.0
for i in range(N):
    # Використовуємо бібліотечну функцію (оператор **)
    sequential_sum += a_full[i] * (x_val ** (i + 1))

print(f"Послідовний результат: {sequential_sum:.10f}")

# Порівняння
diff = abs(parallel_res - sequential_sum)
print(f"Різниця (похибка): {diff:.2e}")
print("-" * 30)

if diff < 1e-7:
    print("Перевірка пройдена успішно! Все на славу Божу.")
else:
    print("Увага: виявлено значну розбіжність.")

```

Результати

```

-----
Результат MPI (1 проц.): 868.5566056458
Послідовний результат:      868.5566056458
Різниця (похибка):          9.09e-13
-----
Перевірка пройдена успішно! Все на славу Божу.

```

4.2.2.2 Паралельні

Завдання

Використовуючи наведені вище програми, розробіть власну програму, в якій між паралельними процесами розподіляється однакова

кількість членів ряду $\sum_{i=1}^N a_i x_i^i$.

Код

```

from mpi4py import MPI
import numpy as np
import time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

```

```

size = comm.Get_size()

# Параметри задачі
N = 10_000
x_val = 1.001

# 1. Визначаємо кількість елементів для кожного процесу (рівномірно)
# Використовуємо divmod для обробки випадків, коли N не ділиться на
size націло
base_n = N // size
remainder = N % size

# Кожен процес отримує base_n елементів, а перші 'remainder' процесів –
ще по одному
local_n = base_n + (1 if rank < remainder else 0)

print(f"{rank}: {local_n}")

# Розрахунок зміщень (displacements) для Scatterv та для обчислення
степенів
sendcounts = np.array(comm.allgather(local_n))
displs = np.insert(np.cumsum(sendcounts[:-1]), 0, 0)
local_displ = displs[rank]

a_full = None
if rank == 0:
    # Генеруємо коефіцієнти
    a_full = np.random.rand(N).astype(np.float64)
    print(f"Running on {size} processors for N={N}")
    print(f"Distribution of elements (even): {sendcounts}")

# Виділяємо буфер для локальної частини масиву
local_a = np.empty(local_n, dtype=np.float64)

# Розподіляємо дані
comm.Scatterv([a_full, sendcounts, displs, MPI.DOUBLE], local_a,
root=0)

# --- ПАРАЛЕЛЬНЕ ОБЧИСЛЕННЯ (з вимірюванням часу) ---
start_time = time.time()

local_sum = 0.0
for i in range(local_n):
    global_idx = local_displ + i + 1
    # Обчислення степеня шляхом перемноження
    power_val = 1.0
    for _ in range(global_idx):
        power_val *= x_val
    local_sum += local_a[i] * power_val

end_time = time.time()
local_duration = end_time - start_time

# Збираємо результати та час роботи кожного процесора
total_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)
all_times = comm.gather(local_duration, root=0)

# --- ПЕРЕВІРКА ТА АНАЛІЗ ---

```

```

if rank == 0:
    print("-" * 40)
    print(f"Parallel sum: {total_sum:.10f}")

    # Послідовна перевірка через бібліотечну функцію
    seq_sum = sum(a_full[i] * (x_val ** (i + 1)) for i in range(N))
    print(f"Successive sum: {seq_sum:.10f}")
    print(f"Division:          {abs(total_sum - seq_sum):.2e}")
    print("-" * 40)

    # Аналіз нерівномірності навантаження
    print("Time of work for each process (sec):")
    for r, t in enumerate(all_times):
        print(f"Rank {r}: {t:.6f} s")

    max_t = max(all_times)
    min_t = min(all_times)
    print(f"\nUnevenness: {(max_t - min_t) / max_t * 100:.2f}%")

```

Результати

```

1: 2500
2: 2500
3: 2500
0: 2500
Running on 4 processors for N=10000
Distribution of elements (even): [2500 2500 2500 2500]
-----
Parallel sum: 11101608.6948592309
Successive sum: 11101608.6948591731
Division:          5.77e-08
-----
Time of work for each process (sec):
Rank 0: 0.249751 s
Rank 1: 0.921180 s
Rank 2: 1.310935 s
Rank 3: 1.788271 s

Unevenness: 86.03%

```

4.2.2.3 Виміри

Завдання

Виміряйте час роботи програми для трьох, досить великих значень кількості членів ряду і порівняйте його з часом вирішення цієї задачі за допомогою вихідної програми даного завдання для двох, трьох та чотирьох процесорів. Результати наведіть у вигляді таблиці.

Kod

```
from mpi4py import MPI
import numpy as np
import math

def get_balanced_layout(N, size):
    """Розрахунок збалансованого розподілу (Вихідна програма)"""
    sendcounts = np.zeros(size, dtype=int)
    displs = np.zeros(size, dtype=int)
    total_ops = N * (N + 1) / 2
    target_load = total_ops / size
    curr_n1, assigned = 1, 0
    for i in range(size - 1):
        a, b, c = 0.5, curr_n1 - 0.5, -target_load
        m_i = int(round((-b + math.sqrt(b**2 - 4 * a * c)) / (2 * a)))
        if assigned + m_i > N:
            m_i = max(0, N - assigned)
        sendcounts[i], displs[i] = m_i, assigned
        assigned += m_i
        curr_n1 += m_i
    sendcounts[size - 1] = N - assigned
    displs[size - 1] = assigned
    return sendcounts, displs

def get_equal_layout(N, size):
    """Розрахунок рівного розподілу елементів (Власна програма)"""
    base_n = N // size
    rem = N % size
    sendcounts = np.array([base_n + (1 if i < rem else 0) for i in
range(size)])
    displs = np.insert(np.cumsum(sendcounts[:-1]), 0, 0)
    return sendcounts, displs

def compute_sum(local_n, local_displ, x_val):
    """Ядро обчислень: сума ряду з ручним множенням степенів"""
    l_sum = 0.0
    for i in range(local_n):
        global_idx = local_displ + i + 1
        p_val = 1.0
        for _ in range(global_idx):
            p_val *= x_val
        l_sum += 1.0 * p_val # Припускаємо a_i = 1.0 для чистоти тесту
    return l_sum

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()
```

```

# Значення N для тестування
N_values = [10000, 15000, 20000]
x_val = 1.0001

if rank == 0:
    print(f"{'=' * 60}")
    print(f"ТЕСТУВАННЯ ПРОДУКТИВНОСТІ (Процесорів: {size})")
    print(f"{'N':>10} | {'Метод':>15} | {'Час (сек)':>12} | {'Результат':>15}")
    print(f"{'-' * 60}")

for N in N_values:
    for mode in ["balanced", "equal"]:
        # Визначаємо структуру розподілу
        if mode == "balanced":
            scounts, displs = get_balanced_layout(N, size)
            mode_name = "Збалансований"
        else:
            scounts, displs = get_equal_layout(N, size)
            mode_name = "Рівномірний"

        local_n = scounts[rank]
        local_displ = displs[rank]

        # Синхронізація перед вимірюванням
        comm.Barrier()
        start_t = MPI.Wtime()

        # Виконання обчислень
        local_res = compute_sum(local_n, local_displ, x_val)

        # Збір результатів
        final_sum = comm.reduce(local_res, op=MPI.SUM, root=0)
        duration = MPI.Wtime() - start_t
        max_duration = comm.reduce(duration, op=MPI.MAX, root=0)

        if rank == 0:
            print(
                f"{N:10d} | {mode_name:15} | {max_duration:12.6f} | {final_sum:15.2e}"
            )

        if rank == 0:
            print(f"{'-' * 60}")

if __name__ == "__main__":
    main()

```

Результати

```

=====
ТЕСТУВАННЯ ПРОДУКТИВНОСТІ (Процесорів: 1)
      N |                Метод |          Час (сек) |          Результат
-----

```

10000	Збалансований		1.286740		1.72e+04
10000	Рівномірний		1.291332		1.72e+04

15000	Збалансований		2.865245		3.48e+04
15000	Рівномірний		2.864207		3.48e+04

20000	Збалансований		5.244350		6.39e+04
20000	Рівномірний		5.097236		6.39e+04

Таблиця

Кількість процесорів	Число N	Метод	Час
1	10 000	Збалансований	1.276870
		Рівномірний	1.269891
	15 000	Збалансований	2.851189
		Рівномірний	2.860192
	20 000	Збалансований	5.104929
		Рівномірний	5.195640
2	10 000	Збалансований	0.652599
		Рівномірний	0.968025
	15 000	Збалансований	1.478876
		Рівномірний	2.194755
	20 000	Збалансований	2.637790
		Рівномірний	3.850634
4	10 000	Збалансований	0.344768
		Рівномірний	0.560991
	15 000	Збалансований	0.764072
		Рівномірний	1.294998
	20 000	Збалансований	1.406052
		Рівномірний	2.320474

Графік

На графіку наведено різницю між двома підходами до розподілу задач.

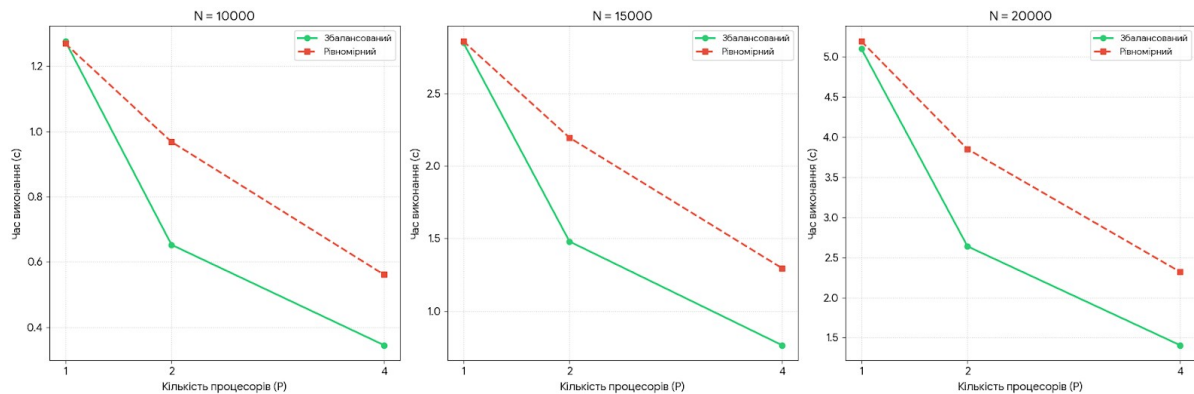


Рисунок 1.4 — Порівняння методів

ПИТАННЯ

Чи можна оператор $M=N/P$ в програмі першого завдання виконати у всіх процесах? Якщо так, покажіть, що треба змінити в програмі.

Так, можна. Це буде навіть більш ефективним, бо позбавляє необхідності пересилати це значення іншим процесам.

Зміни в коді:

```
# Замість comm.scatter(sendcounts, root=0) написати:  
size = comm.Get_size()  
local_N = N // size  
if rank < (N % size):  
    local_n += 1
```

Використовуючи довідкові матеріали на комп'ютері, самостійно вивчіть функцію `MPI_Gatherv()`.

Збирає дані різної довжини з усіх процесів в один великий масив на головному процесі (root).

Параметри: вимагає масиву *recvcounts* (скільки даних чекаємо від кожного процесу) та *displs* (куди саме в результуючий масив їх класти).

Що таке функція обміну з блокуванням, які функції її виконують в MPI?

Функції обміну з блокуванням не дозволяють програмі рухатися далі, доки операція не буде безпечно завершена.

Основні функції в MPI:

- *MPI_Send*, *MPI_Recv*;
- колективні функції: *MPI_Reduce*, *MPI_Bcast*, *MPI_Scatter*, *MPI_Gather*.

Для чого використовується функція *MPI_Barrier()*?

Функція *MPI_Barrier()* є функцією синхронізації. Вона створює контрольну точку, за яку жоден процес не перейде доки всі процеси не досягнуть заданої точки.

Використання:

- перед початком вимірювання часу (функції *MPI.Wtime()*), щоб усі стартували одночасно;
- перед операціями вводу-виводу.

Як в програмі, що використовує функції MPI, створити приймальний буфер, який у точності відповідає повідомленню, що передається?

Щоб не вгадувати розмір повідомлення, використовується механізм "зондування":

- *MPI_Probe*: Функція перевіряє вхідне повідомлення без його фактичного зчитування. Вона заповнює об'єкт *status*.
- *MPI_Get_count*: Витягує зі *status* точну кількість елементів певного типу.
- Динамічне виділення: Отримавши число, ви створюєте масив потрібного розміру і викликаєте *MPI_Recv*.

Чим відрізняються алгоритми логарифмічного здвоєння та рекурсивного подвоєння?

Обидва алгоритми мають складність $O(\log_2 P)$, але працюють у різних напрямках:

- Логарифмічне здвоєння (Reduction Tree): Дані збираються "до центру" (наприклад, у *MPI_Reduce*).
- Рекурсивне подвоєння (Recursive Doubling): Використовується в *MPI_Allgather* або *MPI_Allreduce*.

Опишіть роботу функції *MPI_Scan()*.

Це функція префіксної операції (або "накопичувального підсумку").

Кожен процес з рангом k отримує результат операції (наприклад, суми) над даними від процесів з рангами від 0 до k включно.

Приклад: Якщо у процесів дані $[1, 2, 3, 4]$, то після *MPI_Scan* з операцією SUM вони отримають результати $[1, 3, 6, 10]$.

Як можна розподілити між процесорами обчислення множення матриць?

Для множення матриць $C = A \times B$ можна використати таку стратегію:

- матриця A ділиться на горизонтальні смуги (рядки) між процесорами;
- матриця B копіюється повністю на всі процесори або передається частинами по колу.